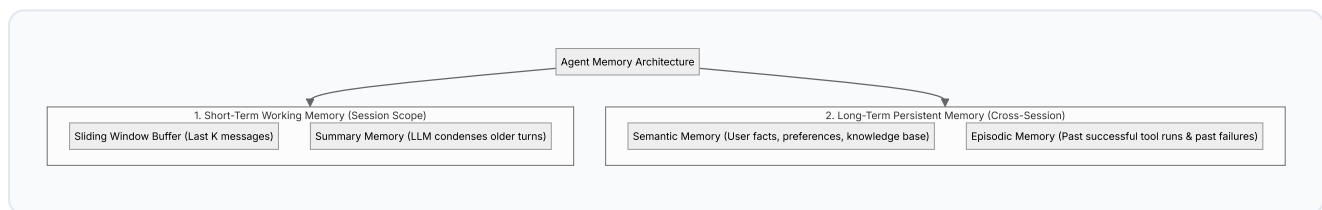


Agentic AI Chapter 6: Memory, Guardrails, LLM Gateways & Evaluations

Core Learning Objective: Productionize AI agents with enterprise-grade resilience. Implement short- and long-term memory architectures, build input/output guardrails against prompt injection, configure multi-provider LLM gateways, and establish automated evaluation frameworks (RAG Triad).

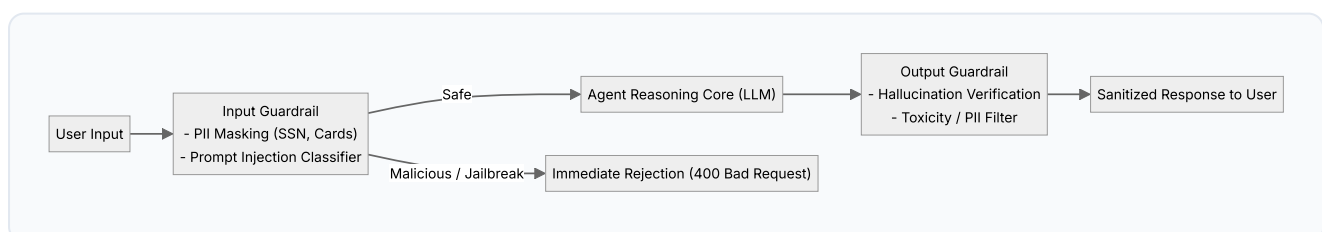
1. The Multi-Tier Memory Hierarchy

An agent without memory treats every user interaction as its first day on earth. Production agents implement a **two-tier memory architecture**:



Implementing Fact-Extraction Long-Term Memory in Python:

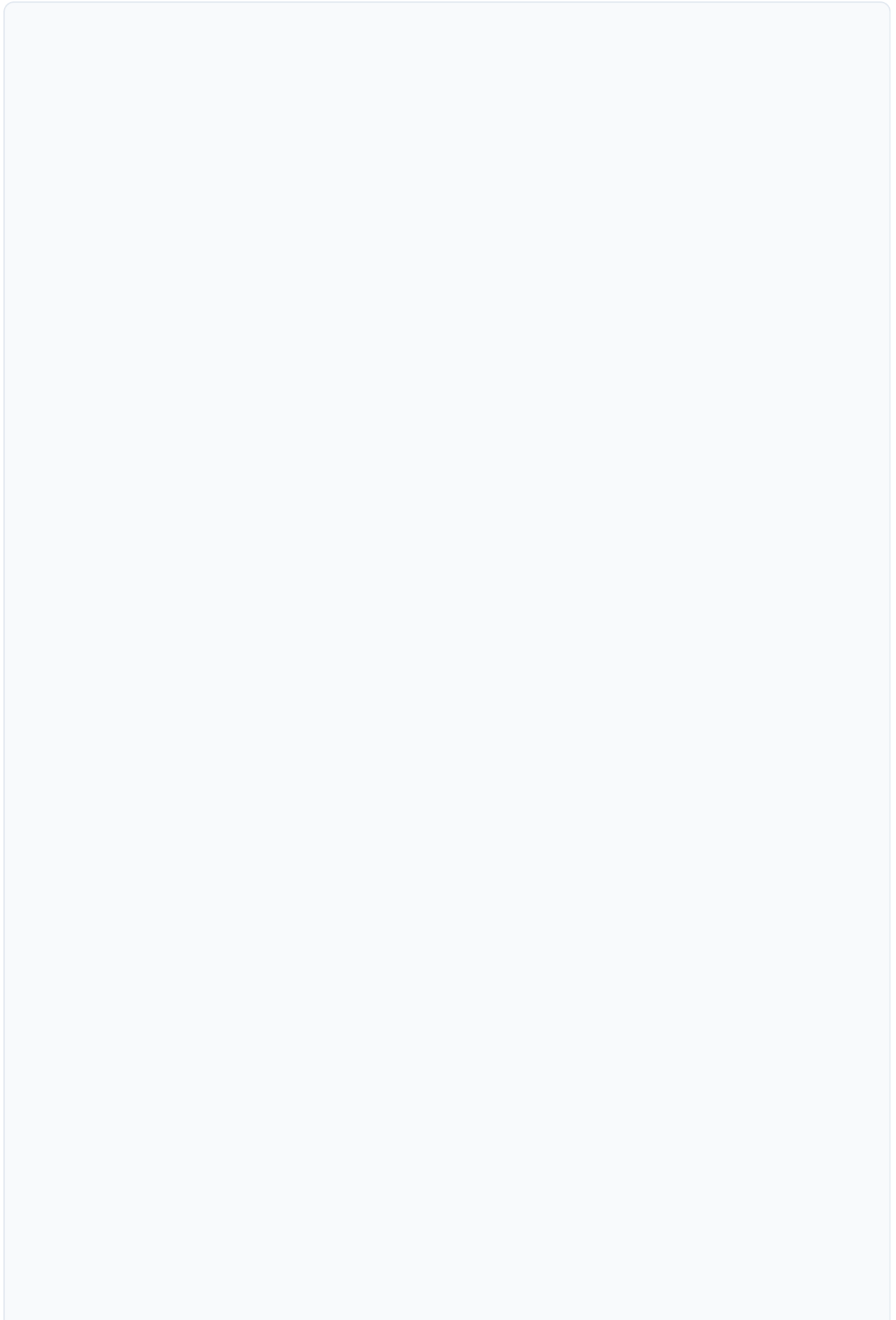
2. Guardrails: Defending Against Prompt Injections & PII Leaks

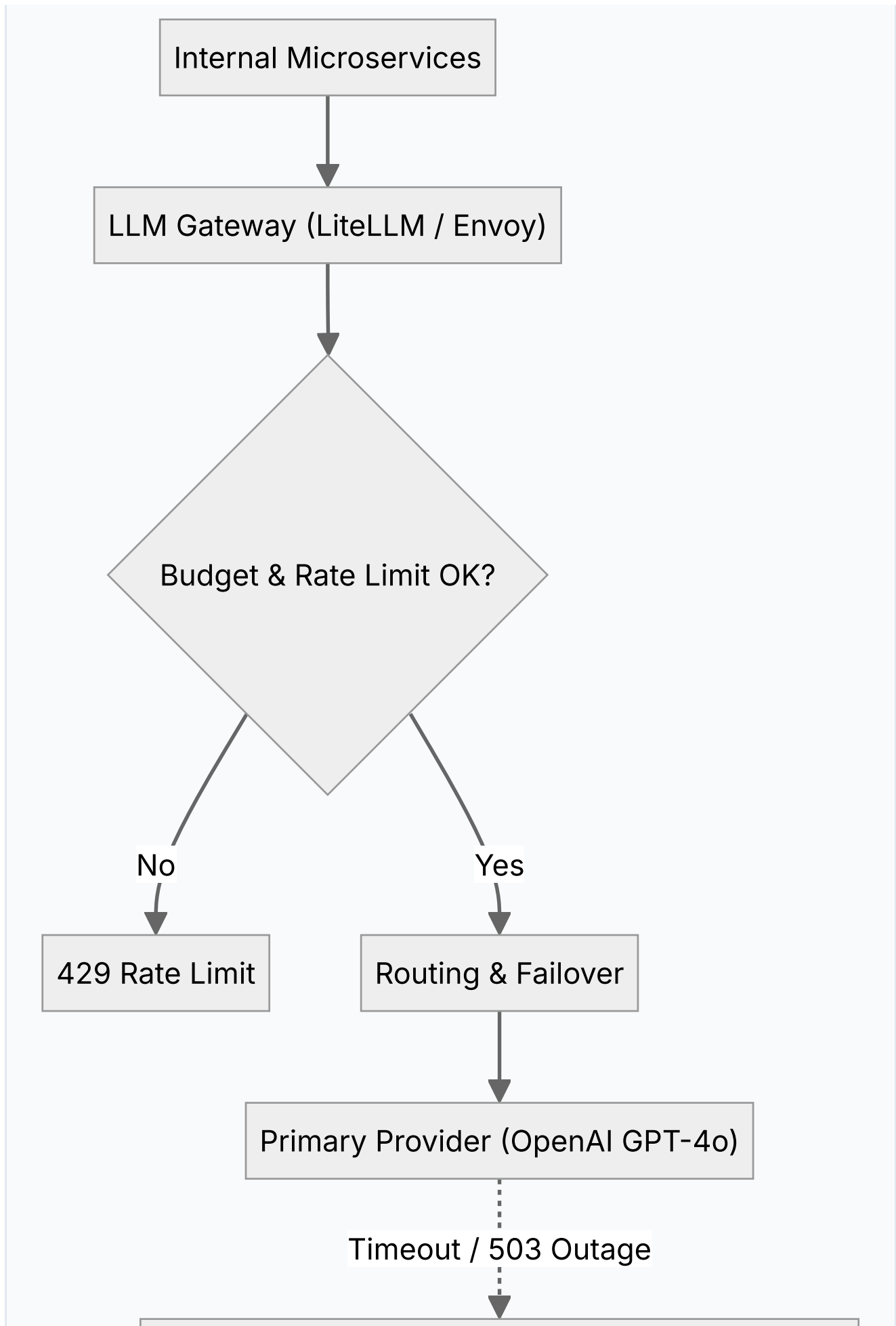


Prompt Injection & PII Redactor:

3. Production LLM Gateways (LiteLLM Architecture)

Never call proprietary model APIs directly in production microservices. Route all traffic through a **Unified LLM Gateway**:





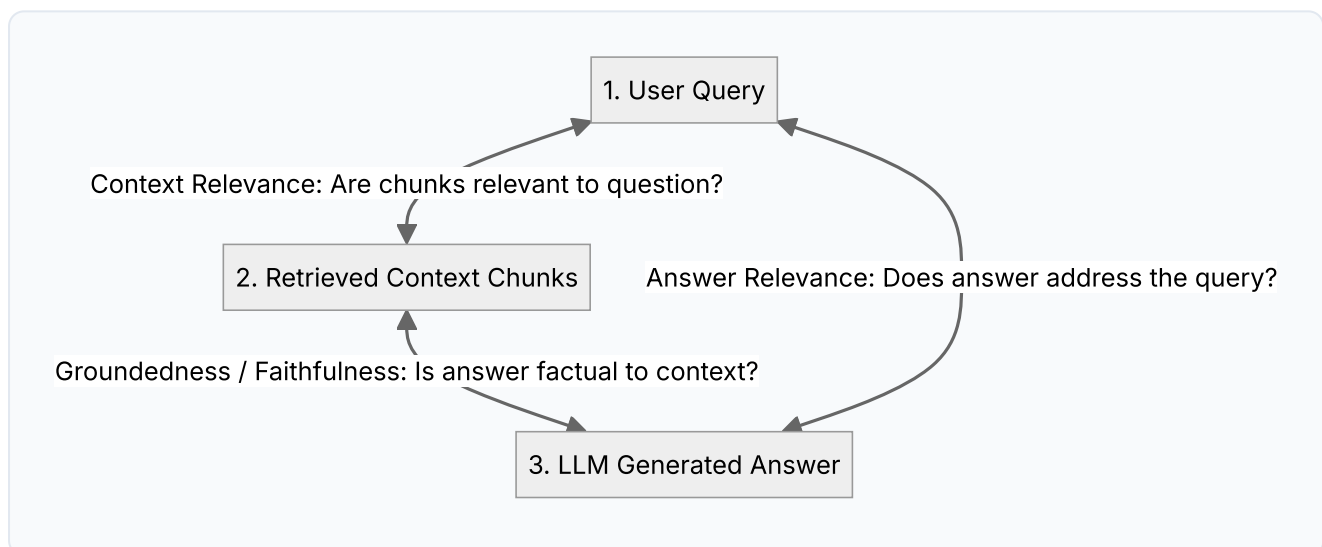
Automatic Fallback (Anthropic Claude 3.5 Sonnet)

Key Gateway Responsibilities:

1. **Multi-Provider Fallback:** If OpenAI returns a 500 or times out, failover instantly to Claude or Gemini with zero user downtime.
2. **Cost & Token Tracking:** Monitor expenditure per department/team in real time.
3. **Semantic Caching:** Cache identical prompts in Redis, reducing API costs by 30-50%.

4. The RAG Evaluation Triad

How do you scientifically benchmark whether your RAG agent is improving? Use the **RAG Triad** (Ragas / TruLens):



1. **Context Relevance:** Evaluates if the retrieved chunks contain the actual facts needed to answer the question without irrelevant noise.
2. **Faithfulness (Groundedness):** Evaluates whether every claim in the generated answer can be directly inferred from the retrieved context (Score 1.0 = zero hallucinations).
3. **Answer Relevance:** Evaluates whether the answer directly and concisely satisfies the user's intent.

4. Runnable Model: Memory Budget, Guardrails and a Policy Gateway

A memory that stays inside a token budget



Input and output guardrails



Regexes catch the obvious cases and are only a first layer: treat them as cheap filters, then add a classifier or a model-based check for what they miss, and never rely on a guardrail prompt alone for security.

A policy gateway: rate limit, spend cap, and an audit trail in one place



A gateway in front of every model call gives you one place for rate limits, budgets, model routing, caching, redaction and audit logs, so a runaway agent or a leaked key is bounded by policy instead of by luck.

Further Reading

- [LiteLLM documentation](#)
- [NeMo Guardrails](#)
- [Generative Agents paper](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. Short-term versus long-term agent memory?
- ▶ 2. What is a guardrail?
- ▶ 3. What does an LLM gateway centralise?
- ▶ 4. Why log prompts and tool calls?